

3 Реализация веб-приложения

3.1 Программная платформа

Для серверной части проекта был выбран язык Golang. Фреймворк GIN [26] – для написания REST API [27], фреймворк gRPC – для взаимодействия сервисов. Для клиентской части была выбрана библиотека React.js. В качестве системы контейнеризации был выбран Docker. Помимо этого, использовались такие операционные системы как Alpine Linux[28], Ubuntu Server[29].

3.2 Система управления базами данных PostgreSQL

Для работы веб-приложения используется PostgreSQL – мощная, надежная и масштабируемая система управления базами данных. Она поддерживает сложные SQL [30]-запросы, транзакции с соответствием стандарту ACID [31]. Скрипт создания представлен в Приложении А.

3.3 Использование библиотеки SQLX

В проекте используется библиотека SQLX [32] для работы с базой данных в Go. SQLX расширяет стандартный пакет database/sql [33], добавляя удобные методы для выполнения SQL-запросов и маппинга результатов на структуры Go, что существенно упрощает процесс разработки кода.

Сопоставление моделей, используемых в SQLX, с их реальными структурами представлено в таблице 3.1.

Таблица 3.1 – Сопоставление моделей, используемых в SQLX

Название модели	Название таблицы
Abonement	abonement
Coach	coach
Order	order
RefreshSessions	refresh_sessions
Review	review
Service	service
User	user
Training	training

				ДП 03.00.ПЗ			
		Ф.И.О.	Подпись	Дата	3 Реализация веб-приложения		
Разраб.	Козляковский Д.А.						
Пров.	Сухорукова И. Г.						
Н. контр.	Сухорукова И. Г.						
Утверд.	Смелов В.В.				БГТУ 1-40 01 01, 2025		
					Лит.	Лист	Листов
					У	1	21

Код, описывающий модель Abonement, приведён в листинге 3.1.

```
type Abonement struct {
    Id          uuid.UUID `db:"id"`
    Title       string    `db:"title"`
    Validity    string    `db:"validity"`
    VisitingTime string    `db:"visiting_time"`
    Photo       string    `db:"photo"`
    Price       int       `db:"price"`
    UpdatedTime time.Time `db:"updated_time"`
    CreatedTime time.Time `db:"created_time"`
    StripePriceId string    `db:"stripe_price_id"`
}
```

Листинг 3.1 – Модель Abonement

Модель Abonement в SQLX описывает абонемент с полями: Id – уникальный идентификатор, Title – название абонемента, Validity – количество месяцев валидности абонемента, VisitingTime – время посещения в течении дня, Photo – фото, Price – цена, UpdatedTime – время обновления абонемента, CreatedTime – время создания абонемента, StripePriceId – цена для продукта из Stripe.

Код, описывающий модель Coach, приведён в листинге 3.2.

```
type Coach struct {
    Id          uuid.UUID `db:"id"`
    Name        string    `db:"name"`
    Description string    `db:"description"`
    Photo       string    `db:"photo"`
    UpdatedTime time.Time `db:"updated_time"`
    CreatedTime time.Time `db:"created_time"}`
```

Листинг 3.2 – Модель Coach

Модель Coach в SQLX описывает тренера с полями: Id – уникальный идентификатор, Name – имя тренера, Description – описание тренера, VisitingTime – время посещения в течении дня, Photo – фото, Price – цена, UpdatedTime – время обновления тренера, CreatedTime – время создания тренера.

Код, описывающий модель Order, приведён в листинге 3.3.

```
type Order struct {Id uuid.UUID `db:"id"` AbonementId uuid.UUID
`db:"abonement_id"`UserId uuid.UUID `db:"user_id"`Status
string`db:"status"`UpdatedTime time.Time `db:"updated_time"`
    CreatedTime time.Time `db:"created_time"`
    ExpiredTime time.Time `db:"expiration_time"}`
```

Листинг 3.3 – Модель Order

Модель Order в SQLX описывает заказ с полями: Id – уникальный идентификатор, AbonementId – уникальный идентификатор абонемента, UserId –

уникальный идентификатор юзера, Status – статус заказа, Photo – фото, Price – цена, UpdatedTime – время обновления заказа, CreatedTime – время создания заказа, ExpiredTime – время истечения валидности заказа.

Код, описывающий модель RefreshSessions, приведён в листинге 3.4.

```
type RefreshSessions struct {
    Id          uuid.UUID `db:"id"`
    UserId      uuid.UUID `db:"user_id"`
    RefreshToken string    `db:"refresh_token"`
    FingerPrint string    `db:"finger_print"`
    CreatedTime time.Time `db:"created_time"`
    UpdatedTime time.Time `db:"updated_time"`
}
```

Листинг 3.4 – Модель RefreshSessions

Модель RefreshSessions в SQLX описывает заказ с полями: Id – уникальный идентификатор, UserId – уникальный идентификатор юзера, RefreshToken – рефреш токен, FingerPrint – отпечаток браузера пользователя, Photo – фото, Price – цена, UpdatedTime – время обновления рефреш сессии, CreatedTime – время создания рефреш сессии.

Код, описывающий модель Review, приведён в листинге 3.5.

```
type Review struct {
    Id          uuid.UUID `db:"id"`
    UserId      uuid.UUID `db:"user_id"`
    Body        string    `db:"body"`
    UpdatedTime time.Time `db:"updated_time"`
    CreatedTime time.Time `db:"created_time"`
}
```

Листинг 3.5 – Модель Review

Модель Review в SQLX описывает заказ с полями: Id – уникальный идентификатор, UserId – уникальный идентификатор пользователя, Body – тело отзыва, UpdatedTime – время обновления отзыва, CreatedTime – время создания отзыва.

Код, описывающий модель Service, приведён в листинге 3.6.

```
type Service struct {
    Id          uuid.UUID `db:"id"`
    Title       string    `db:"title"`
    Photo       string    `db:"photo"`
    UpdatedTime time.Time `db:"updated_time"`
    CreatedTime time.Time `db:"created_time"}`
```

Листинг 3.6 – Модель Service

Модель Service в SQLX описывает услугу с полями: Id – уникальный идентификатор, Title – название услуги, Photo – фото, UpdatedTime – время

обновления услуги, CreatedTime – время создания отзыва.

Код, описывающий модель User, приведён в листинге 3.7.

```
type User struct {
    ID          uuid.UUID `db:"id"`
    Name        string    `db:"name"`
    PasswordHash string    `db:"password_hash"`
    Email       string    `db:"email"`
    Role        string    `db:"role"`
    Photo       string    `db:"photo"`
    UpdatedTime time.Time `db:"updated_time"`
    CreatedTime time.Time `db:"created_time"`
}
```

Листинг 3.7 – Модель User

Модель User в SQLX описывает пользователя с полями: ID – уникальный идентификатор, Name – имя, PasswordHash – хэшированный пароль, Email – электронная почта, Role – роль, Photo – фото, UpdatedTime – время обновления пользователя, CreatedTime – время создания пользователя.

Код, описывающий модель Training, приведён в листинге 3.8.

```
type Training struct {
    Id          uuid.UUID
    TimeFrom    time.Time
    TimeUntil   time.Time
    Status      string
    CoachId     uuid.UUID
    ClientId    uuid.UUID
    CreatedTime time.Time
    UpdatedTime time.Time
}
```

Листинг 3.8 – Модель Training

Модель Training в SQLX описывает тренировку с полями: Id – уникальный идентификатор, TimeFrom – время начала тренировки, TimeUntil – время окончания тренировки, Status – статус тренировки, CoachId – уникальный идентификатор тренера, ClientId – уникальный идентификатор клиента, UpdatedTime – время обновления тренировки, CreatedTime – время создания тренировки.

3.4 Программные библиотеки

В ходе разработки серверной части веб-приложения, с целью повышения эффективности функционирования системы, были задействованы различные программные библиотеки. Они обеспечили устойчивость, масштабируемость и высокую скорость отклика. Используемые программные библиотеки перечислены в таблице 3.2.

Таблица 3.2 – Программные библиотеки серверной части

Библиотека	Версия	Назначение
Validator [34]	v1.10.0	Библиотека для валидации значений структур и отдельных полей на основе тегов.
CORS gin's middle-ware [35]	v1.7.2	Библиотека, которая предоставляет middleware для GIN для включения поддержки CORS [36].
jwt-go [37]	v5.2.1	Библиотека для работы с JWT [38] (JSON Web Tokens).
uuid [39]	v1.6.0	Библиотека для создания и проверяет UUID [40].
GoDotEnv [41]	v1.5.1	Библиотека для загрузки переменных окружения из файла .env
Pg [42]	v1.10.9	Драйвер для работы с PostgreSQL в Golang.
Go Stripe [43]	v81.1.1	Библиотека для интеграции с платежной системой Stripe, используется для обработки платежей.
Swag [44]	v1.0.1	Библиотека для преобразования аннотации Go в документацию Swagger 2.0 [45]
gin-swagger [46]	v1.6.0	Библиотека, которая предоставляет middleware для GIN для автоматической генерации документации RESTful API с Swagger 2.0.
gRPC-Go [47]	v1.68.0	Библиотека, которая реализует gRPC в Go.
Protobuf [48]	v1.35.1	Библиотека для поддержки Protobuf в Go.
aws-sdk-go-v2 [49]	v1.32.6	Библиотека для интеграции с AWS.
sqlx	v1.4.0	Библиотека для добавления удобных методов для выполнения SQL-запросов и маппинга результатов на структуры Go
Migrate [50]	v4.18.1	Библиотека для работы с миграциями для баз данных.
Crypto [51]	v0.27.0	Библиотека, которая предоставляет криптографические функции и алгоритмы.
gorilla/websocket [52]	v1.5.3	Библиотека, которая предоставляет функционал для работы с websocket.

В процессе разработки клиентской части веб-приложения были задействованы программные библиотеки. Их использование позволило ускорить разработку и обеспечить удобную реализацию пользовательского интерфейса. Программные библиотеки представлены в таблице 3.3.

Таблица 3.3 – Программные библиотеки клиентской части

Библиотека	Версия	Назначение
mui/icons-material [53]	v5.15.14	Библиотека иконок от Material-UI [54], предоставляет набор иконок, для интеграции с React.
mui/material [55]	v5.15.14	Библиотека, предоставляющая готовые компоненты, ориентированные на стилизацию и гибкость.
mui/joy [56]	v5.0.0	Библиотека, предоставляющая готовые компоненты, ориентированные на стилизацию и гибкость.

Продолжение таблицы 3.3

Библиотека	Версия	Назначение
reduxjs/toolkit [57]	v2.2.2	Библиотека для управления состоянием в приложениях с использованием Redux [58].
react-stripe-js [59]	v3.1.1	Библиотека для интеграции с Stripe для работы с платежами, предоставляющая клиентскую сторону для взаимодействия с API Stripe.
Axios [60]	v1.6.8	Библиотека для выполнения HTTP-запросов.
React	v18.2.0	Библиотека для создания пользовательских интерфейсов, используется для построения компонентов и управления состоянием UI
react-router-dom [61]	v6.22.3	Библиотека для маршрутизации в React-приложениях.
Notistack [62]	v3.0.1	Библиотека – менеджер уведомлений для React-приложений.

Программные библиотеки позволяют упростить реализацию веб-приложения.

3.5 Сторонние сервисы Stripe, LocalStack

Для успешной реализации функционала веб-приложения были использованы следующие сторонние сервисы:

- Stripe – интеграция с этим сервисом позволила реализовать обработку онлайн-платежей в приложении.
- LocalStack – используется для имитации работы сервисов AWS. Интеграция с входящим в него s3 использовалась для хранения фотографий.

3.6 Структура серверной части

Структура серверной части основывается на микросервисной архитектуре.

За основу были приняты паттерн API Gateway [63] и принцип разделения сервисов, по которому каждый сервис отвечает за отдельную сущность или процесс в веб-приложениях. API Gateway представлен сервисом, который представляет из себя REST API сервер принимающий HTTP запросы и преобразующий их в Protobuf формат для дальнейшей отправки на другие сервисы. Еще в нем есть механизмы аутентификации пользователей. А также методы работы с платежным шлюзом.

Основные компоненты структуры Gateway сервиса включают в себя несколько ключевых элементов, которые обеспечивают эффективную работу приложения:

- маршрутизаторы – управляют маршрутами и направляют запросы к

соответствующим контроллерам;

- контроллеры – обрабатывают запросы от клиента, преобразуют их в формат Protobuf для дальнейшей отправки в сервисы и возвращают ответы;
- middleware – промежуточные обработчики, используемые для валидации данных и обеспечения безопасности.

В таблице 3.4 приведён список директорий серверной части проекта.

Таблица 3.4 – Директории Gateway сервиса

Директория	Назначение
cmd	Является точкой входа в приложение. Подгружает переменные окружения из .env для удобного последующего обращения к ним. Запускает метод инициализации структуры всего сервиса. Запускает метод запуска сервиса на порту.
docs	Содержит файлы для swagger-документации.
internal	Содержит пакеты доступные только внутри текущего проекта. Не могут быть импортированы из другого проекта.
pkg	Содержит общедоступные переиспользуемые пакеты – логгер.
Internal/server	Содержит структуру всего сервиса, методы ее инициализации.
internal/<название сервиса>	Содержит каталоги для работы с сервисом
internal/<название сервиса>/rest	Содержит объявление контроллеров и метод связывания маршрутов с контроллерами.
internal/<название сервиса>/middlewares	Содержит middlewares специфичные для конкретного сервиса.
internal/<название сервиса>/dtos	Содержит data transfer objects специфичные для конкретного сервиса.
internal/<название сервиса>/<название сервиса>_errors	Содержит ошибки специфичные для конкретного сервиса.
internal/common_middlewares	Содержит общие middlewares: аутентификация, авторизация и ошибки.

Таблица соответствия маршрутов контроллерам представлена в таблице 3.5.

Таблица 3.5 – Контроллеры и функции маршрутов

Метод	Маршрут	Контроллер	Номер из диаграммы вариантов использования	Описание
1	2	3	4	5
GET	/abonements	GetAbonements	9	Возвращает абонементы с их услугами.
POST	/abonements	CreateAbonement	15	Передаёт данные для создания абонемента и связей с услугами сервисам, возвращает созданный абонемент.

Продолжение таблицы 3.5

1	2	3	4	5
PUT	/abonements	UpdateAbonement	17	Передаёт данные для обновления абонемента и связей с услугами другим сервисам, возвращает обновленный абонемент с его услугами.
DELETE	/abonements/:id	DeleteAbonement	16	Передаёт данные для удаления абонемента и его связей с услугами другим сервисам, возвращает удаленный абонемент.
GET	/coaches	GetCoaches	11	Возвращает тренеров с их услугами, отзывами и пользователями.
POST	api/delete-product	CreateCoach	19	Передаёт данные для создания тренера и связей с услугами другим сервисам, возвращает созданного тренера с его услугами.
PUT	api/product-details	UpdateCoach	21	Передаёт данные для обновления тренера и связей с услугами другим сервисам, возвращает обновленного тренера с его услугами.
DELETE	/coaches/:id	DeleteCoach	20	Передаёт данные для удаления тренера другим сервисам, возвращает удаленного тренера.
POST	/checkout-session-complet	HandleCheckoutSessionCompleted	3	Обработывают данные о успешно пройденной оплате, передают данные для создания заказа другим сервисам.
POST	/create-checkout-session	CreateCheckoutSession	3	Передают данные для создания stripe checkout сессии другим сервисам, возвращает URL созданной stripe checkout сессии.
GET	/orders/:userid	GetUserOrders	6	Передают данные для возврата заказов клиента другим сервисам, возвращает заказы клиента.
POST	/reviews	CreateCoachReview	4	Передаёт данные для создания отзыва и связи с тренером другим всевозможными сервисам.

Завершение таблицы 3.5

1	2	3	4	5
DELETE	/reviews/:id	DeleteCoachReview	26	Удаляет отзыв к тренеру.
POST	/sso/signUp	SignUp	1	Передаёт данные для регистрации другим сервисам, устанавливают в куки [64] refreshToken, возвращает созданного клиента и accessToken [65].
POST	/sso/signIn	SignIn	2	Передаёт данные для входа другим сервисам, устанавливают в куки refreshToken, возвращает созданного клиента и accessToken.
PUT	/users/:id	UpdateUser	5	Передаёт данные для обновления пользователя другим сервисам, возвращает обновленного пользователя.
GET	/users	GetClients	13	Возвращает всех клиентов.
DELETE	/users/:id	DeleteClientById	13	Передаёт данные для удаления клиента другим сервисам, возвращает удаленного клиента.

При передаче данных между клиентом и Gateway сервисом используется формат JSON (JavaScript Object Notation) [66].

Микросервисная архитектура представлена сервисами для работы с отдельными частями приложения.

В таблице 3.6 приведён список сервисов и их назначение.

Таблица 3.6 – Сервисы серверной части веб-приложения

Сервис	Назначение
SSO	Реализует единую точку входа в приложение. Реализует функции регистрации, входа, выхода и работу с токенами access и refresh.
Migrate	Нужен для начальной инициализации схемы базы данных и для оповещения других сервисов о том, что схема базы данных находится в последней версии.
Abonement	Реализует логику работы с абонентами
User	Реализует логику работы с пользователями
Review	Реализует логику работы с отзывами
Order	Реализует логику работы с заказами
Coach	Реализует логику работы с тренерами
Service	Реализует логику работы с услугами

Каждый сервис представляет из себя gRPC сервис.

Основные компоненты структуры сервиса включают в себя несколько элементов, которые обеспечивают эффективную работу веб-приложения:

– методы – принимают запросы от клиентов, выполняют бизнес-логику

через слой Usecase и возвращают ответы;

- usecase – содержат бизнес-логику, обрабатывают данные и взаимодействуют с репозиториями;

- repository – занимаются взаимодействием с базой данных.

В таблице 3.7 приведён список директорий сервиса и назначение файлов, хранящихся в этих директориях.

Таблица 3.7 – Директории сервиса

Директория	Назначение
cmd	Является точкой входа в приложение. Подгружает переменные окружения из .env для удобного последующего обращения к ним. Запускает метод инициализации структуры всего сервиса. Запускает метод запуска сервиса на порту.
internal	Содержит пакеты доступные только внутри текущего проекта. Не могут быть импортированы из другого проекта.
pkg	Содержит общедоступные переиспользуемые пакеты – логгер.
internal/server	Содержит структуру всего сервиса, методы ее инициализации, а также метод запуска сервиса.
internal/delivery	Содержит структуру gRPC сервера, метод ее инициализации и методы обработки запросов, которые вызывают слой usecase.
internal/usecase	Содержит интерфейсы и их реализацию для usecase, которые выполняют бизнес логику. Usecase вызывают слой repository.
internal/repository	Содержит интерфейсы и их реализацию для repository, которые работают с базой данных.
internal/models	Содержит модели для работы с базой данных.
internal/dtos	Содержит структуры для переноса данных их слоя обработчиков, в слой usecase.
internal/constants	Содержит общие константы сервиса.
internal/errors	Содержит общие ошибки сервиса.

Protobuf для сервиса SSO представлен в приложении Б.

Соответствие gRPC методов сервиса абонементов с их описанием представлено в таблице 3.8.

Таблица 3.8 – Таблица соответствия gRPC методов сервиса абонементов с их описанием

Метод	Назначение
CreateAbonement	Создает абонемент, его связи с услугами, stripe продукт, stripe цену. Сохраняет фото в s3 localstack. Возвращает абонемент со связанными услугами.
GetAbonementById	Возвращает абонемент по введенному id.
UpdateAbonement	Обновляет абонемент, его связи с услугами, stripe цен, фото в s3 localstack.

Продолжение таблицы 3.8

Метод	Назначение
DeleteAbonementById	Удаляет абонемент, его связи с услугами, фото в s3 localstack. Архивирует stripe продукт. Возвращает удаленный абонемент.
GetAbonements	Возвращает все абонементы.
GetAbonementsWithServices	Возвращает все абонементы со связанными с ними услугами.
GetAbonementsByIds	Возвращает абонементы по введенным id`s.

Соответствие gRPC методов сервиса тренеров с их описанием представлено в таблице 3.9.

Таблица 3.9 – Таблица соответствия gRPC методов сервиса тренеров с их описанием

Метод	Назначение
CreateCoach	Создает тренера, его связи с услугами. Сохраняет фото в s3 localstack. Возвращает тренера со связанными услугами.
GetCoachById	Возвращает тренера по введенному id.
UpdateCoach	Обновляет тренера, его связи с услугами, фото в s3 localstack. Возвращает тренера со связанными услугами.
DeleteCoachById	Удаляет тренера, его связи с услугами, фото в s3 localstack. Возвращает удаленного тренера.
GetCoaches	Возвращает всех тренеров.
GetCoachesWithServicesWithReviewsWithUsers	Возвращает всех тренеров с относящимися к ним услугам, с относящимися к ним отзывами и клиентами, оставившими их.

Соответствие gRPC методов сервиса заказов с их описанием представлено в таблице 3.10.

Таблица 3.10 – Таблица соответствия gRPC методов сервиса заказов с их описанием

Метод	Назначение
CreateOrder	Создает заказ. Возвращает заказ.
GetUserOrders	Возвращает все заказы клиента.
CreateCheckoutSession	Создает Stripe checkout сессию. Возвращает ее URL.

Соответствие gRPC методов сервиса отзывов с их описанием представлено в таблице 3.11.

Таблица 3.11 – Таблица соответствия gRPC методов сервиса отзывов с их описанием

Метод	Назначение
CreateCoachReview	Создает отзыв для тренера. Возвращает отзывы тренера.
GetReviewById	Возвращает отзыв по введенному id.

Продолжение таблицы 3.11

Метод	Назначение
UpdateReview	Обновляет отзыв. Возвращает обновленный отзыв.
DeleteReviewById	Удаляет отзыв по введенному id. Возвращает удаленный отзыв.
GetCoachReviews	Возвращает отзывы тренера.
GetCoachesReviews	Возвращает отзывы тренеров.

Сервис для предоставления услуг, либо же сервисов, включает в себя API для работы с сущностью «услуга фитнес-центра». Данное API реализовано с использованием технологии gRPC и предоставляет функциональность для создания, получения, удаления и выполнения иных операций над объектами.

Разработка API велась с учётом требований к масштабируемости, производительности и удобству интеграции с другими компонентами системы. Использование gRPC дало высокую скорость обмена данными.

Соответствие gRPC-методов сервиса услуг с их описанием представлено в таблице 3.12.

Таблица 3.12 – Таблица соответствия gRPC методов сервиса услуг с их описанием

Метод	Назначение
CreateService	Создает услугу, сохраняет фото в s3 localstack. Возвращает созданную услугу.
GetServiceById	Возвращает услугу по введенному id.
UpdateService	Обновляет услугу, обновляет фото в s3 localstack. Возвращает обновленную услугу.
DeleteServiceById	Удаляет услугу по введенному id, удаляет фото из s3 localstack. Возвращает удаленную услугу.
GetServices	Возвращает все услуги.
CreateCoachServices	Создает связи услуг с тренером. Возвращает связи тренера с услугами.
CreateAbonementServices	Создает связи услуг с абонементом. Возвращает связи абонемента с услугами.
UpdateAbonementServices	Обновляет связи услуг с абонементом. Возвращает обновленные связи абонемента с услугами.
UpdateCoachServices	Обновляет связи услуг с тренером. Возвращает обновленные связи тренера с услугами.
GetAbonementsServices	Возвращает услуги абонементов.
GetCoachesServices	Возвращает услуги тренеров.

Соответствие gRPC методов сервиса авторизации с их описанием представлено в таблице 3.13.

Таблица 3.13 – Таблица соответствия gRPC методов сервиса услуг с их описанием

Метод	Назначение
SignUp	Создает клиента, refresh сессию, access и refresh токены. Возвращает клиента, access и refresh токены.
SignIn	Идентифицирует пользователя, обновляет refresh сессию, access и refresh токены. Возвращает пользователя, access и refresh токены.
LogOut	Удаляет refresh сессию.
Refresh	Обновляет refresh сессию, access и refresh токены. Возвращает пользователя, access и refresh токены.

Соответствие gRPC методов сервиса пользователя с их описанием представлено в таблице 3.14.

Таблица 3.14 – Таблица соответствия gRPC методов сервиса пользователя с их описанием

Метод	Назначение
CreateUser	Создает пользователя, сохраняет фото в s3 localstack. Возвращает созданного пользователя.
GetUserById	Возвращает пользователя по введенному id.
UpdateUser	Обновляет пользователя, фото в s3 localstack. Возвращает обновленного пользователя.
DeleteUserById	Удаляет пользователя по введенному id. Возвращает удаленного пользователя.
GetUserByEmail	Возвращает пользователя по введенному email.
CheckPassword	Проверяет пароль введенный пароль на валидность.
GetUsersByIds	Возвращает пользователей по id`s.
GetClients	Возвращает всех клиентов.

При передаче данных между сервисами используется формат Protobuf.

Имеется также сервис для онлайн тренировок. Таблица соответствия маршрутов контроллерам в исходном коде представлена в таблице 3.15.

Таблица 3.15 – Контроллеры и функции маршрутов

Метод	Маршрут	Контроллер	Номер из диаграммы вариантов использования	Описание
POST	/training/book	Insert	26	Создает объект тренировки, сохраняет ее в базу данных postgres.
DELETE	/training/id	Delete	27	Удаляет запись тренировки из базы данных postgres.
GET	/training/join/:roomId	Join	28	Создает объект комнаты. Сохраняет его в хранилище на уровне приложения. Отвечает за пересылку данными для установления WebRTC.

Продолжение таблицы 3.15

Метод	Маршрут	Контроллер	Номер из диаграммы вариантов использования	Описание
GET	/training/:day/:id	GetTrainingsById	25	Возвращает тренировки за день по тренеру.

Сервис для онлайн тренировок предоставляет API в формате REST для управление сущностью тренировок. А также несет в себе логику управление жизненным циклом онлайн-тренировок.

3.7 Реализация сквозных функций пользователей

3.7.1 Регистрация

Для пользователя доступна регистрация, которая позволяет ему создать учетную запись в системе. Этот процесс реализован в контроллере SignUp

Он принимает HTTP-запрос с данными из тела, включая email, имя, пароль. Проецирует данные на структуру для получения конкретного типа данных. Валидирует полученную структуру. Дополнительно, из контекста запроса извлекается параметр fingerprint, который используется для повышения безопасности токенов. Создает запрос в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод SignUp у сервиса SSO, который руководит создание клиента, Refresh сессии, refresh и access токенов.

В результате выполнения метода возвращаются accessToken, созданный клиент и refreshToken устанавливается в куки.

3.7.2 Аутентификация

Для пользователя доступна аутентификация. Она основывается на подходе с парой jwt токенов. При входе в приложение при помощи контроллера SignIn.

Он принимает HTTP-запрос с данными из тела, включая email, пароль. Проецирует данные на структуру для получения конкретного типа данных. Валидирует полученную структуру. Этот этап играет ключевую роль в обеспечении безопасного и персонализированного доступа к системе. Процесс авторизации реализован с учётом современных требований к защите пользовательских данных. Дополнительно, из контекста запроса извлекается параметр fingerprint, который используется для повышения безопасности токенов. Создает запрос в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод SignIn у сервиса SSO, который руководит идентификацией пользователя, обновлением Refresh сессии, созданием refresh и access токенов.

В результате выполнения метода возвращаются accessToken, пользователь, refreshToken устанавливается в куки.

3.8 Реализация функций доступных пользователям с ролями «Клиент» и «Администратор»

3.8.1 Просмотр информации о абонементных

У клиентов и администраторов есть возможность просмотра информации о абонементных. Эта возможность реализована в контроллере GetAbonements.

Этот контроллер обращается к методу GetAbonementsWithServices для получения всех абонементов с относящимися к ним услугами.

В результате выполнения возвращаются абонементы с относящимися к ним услугами.

3.8.2 Поиск, сортировка, фильтрация абонементов

У клиентов и администраторов есть возможность поиска, сортировки, фильтрации абонементов. Эта возможность реализована в методах filterData и sortAbonnements на клиентской части.

Метод filterData фильтрует абонементы по имени, периоду валидности, времени посещения, услугам и цене.

Исходный код метода sortAbonnements приведен в листинге 3.5.

```
const sortAbonnements = (abonnements, sortingFilter, order) =>
{
  return abonnements.slice().sort((a, b) => {
    if (sortingFilter === "price") {return order === "asc"
      ? a.abonement.price - b.abonement.price
      : b.abonement.price - a.abonement.price;    }
    if (sortingFilter === "title") {
const titleA = a.abonement.title.toLowerCase();const titleB =
b.abonement.title.toLowerCase();if (order === "asc") {
      return titleA.localeCompare(titleB); } else {
      return titleB.localeCompare(titleA);
    }}return 0;});};
```

Листинг 3.9 – Реализация метода sortAbonnements

Метод sortAbonnements сортирует абонементы по цене или названию.

3.8.3 Просмотр информации о тренерах и отзывов о тренерах

У клиентов и администраторов есть возможность просмотра информации о тренерах и отзывов о тренерах. Эта возможность реализована в контроллере GetCoaches.

Этот контроллер обращается к методу Get Coaches With Services With Reviews With Users для получения всех тренеров с их услугами и с отзывами, а также с пользователями, которые их оставили.

В результате выполнения возвращаются все тренера с их услугами и с отзывами, а также с пользователями, которые их оставили.

3.9 Реализация функций для пользователя с ролью «Клиент»

3.9.1 Оставить отзыв о тренере

У клиентов есть возможность оставить отзыв о тренере. Эта возможность реализована в контроллере `CreateCoachReview`.

Он принимает HTTP-запрос с данными из тела `id` пользователя, тело отзыва, `id` тренера. Проецирует данные на структуру для получения конкретного типа данных. Валидирует полученную структуру. Дополнительно, из контекста запроса извлекается параметр `fingerprint`, который используется для повышения безопасности токенов. Создает запрос в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод `CreateCoachReview` у сервиса `Review`, который руководит созданием отзыва к тренеру. Вызывает метод `GetUserById` для взяти пользователя по `id` пользователя.

В результате выполнения контроллера возвращаются отзыв и пользователь написавший его.

3.9.2 Покупка абонементов

У клиентов есть возможность покупки абонементов. Эта возможность реализована в контроллере `CreateCheckoutSession`.

Реализация этого контроллера приведена в листинге 3.9.

```
func (h *Handler) CreateCheckoutSession(c *gin.Context) {
    ccsDto := &dtos.CreateCheckoutSessionDTO{}
    if err := c.ShouldBindJSON(&ccsDto); err != nil {
        logger.ErrorLogger.Printf("Error binding
CreateCheckoutSessionRequest: %v", err)
        c.JSON(http.StatusBadRequest, gin.H{"error":err.Error()})
        return}
    createCheckoutSessionRequest :=
&orderGRPC.CreateCheckoutSessionRequest{ClientId:
ccsDto.ClientId, AbonementId: ccsDto.AbonementId, StripePriceId:
=(*h.orderClient).CreateCheckoutSession(context)
```

Листинг 3.10 – Реализация контроллера `CreateCheckoutSession`

Он принимает HTTP-запрос с данными из тела: `id` клиента, `id` абонемента, `id` цены в stripe. Проецирует данные на структуру для получения конкретного типа данных. Создает запрос в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод `CreateCheckoutSession` у сервиса `Order`, руководящий созданием stripe checkout сессии, приведен в приложении В.

В результате выполнения контроллера возвращаются URL stripe checkout сессии. Клиента перенаправляет на этот URL, где он может заполнить данные своей карты и оплатить абонемент.

После успешной оплаты Stripe посылает событие об успешном завершении stripe checkout на Gateway service, с обработкой на `HandleCheckoutSessionCompleted`, приведен в приложении Г. После чего Gateway service инициирует создание объекта заказа в Order service.

3.9.3 Редактирование профиля

У клиентов есть возможность редактирования профиля. Эта возможность реализована в контроллере UpdateUser.

Он принимает HTTP-запрос с данными из тела: id клиента, имя, фото. Проецирует данные на структуру для получения конкретного типа данных. Валидирует данные. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод Send у сервиса User, который руководит обновлением пользователя. В результате выполнения контроллера возвращаются обновленный пользователь.

3.9.4 Просмотр списка своих заказов

У клиентов есть возможность просмотра списка своих заказов. Эта возможность реализована в контроллере GetUserOrders.

Он принимает HTTP-запрос с id пользователя. Проецирует данные на структуру для получения конкретного типа данных. Валидирует данные. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод GetUserOrders у сервиса Order, который возвращает заказы пользователя. В результате выполнения контроллера возвращаются заказы пользователя.

3.9.5 Бронирование онлайн тренировки

У клиентов есть возможность бронирования онлайн тренировки. Эта возможность реализована в контроллере Insert. Контроллер принимает HTTP-запрос, который включает id тренера, id клиента, дату тренировки, время начала тренировки и время её окончания. Затем данные проецируются на структуру для получения конкретного типа данных, после чего осуществляется их валидация для проверки корректности. Контроллер вызывает метод Insert у сервисного слоя, который выполняет логику создания сущности тренировки и записывает ее в базу данных. По итогу возвращает объект тренировки.

За расписанием тренировок следит фоновая задача, которая через заданный интервал времени меняет статусы у тренировок. Что обеспечивает управление жизненным циклом тренировки. У тренировки есть всего 3 состояния: «Забронированная», «Активная», «прошедшая».

3.9.6 Отмена бронирования онлайн тренировки

У клиентов есть возможность бронирования онлайн тренировки. Эта возможность реализована в контроллере Delete.

Он принимает HTTP-запрос с id тренировки. Проецирует данные на структуру для получения конкретного типа данных. Валидирует данные. Вызывает метод Delete у сервисного слоя, который удаляет тренировку.

3.10 Реализация функций для пользователя с ролью «Администратор»

3.10.1 Редактирование клиентов

Администратор системы имеет возможность редактирования клиентов – удаление клиентов.

Он принимает HTTP-запрос с id пользователя. Валидирует данные. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод DeleteUserById у сервиса User, который руководит удалением пользователя.

В результате выполнения контроллера возвращаются удаленный пользователь.

3.10.2 Редактирование абонементов

Администратор системы имеет возможность редактирования абонементов – добавление, обновление, удаление. Данная возможность реализована в рамках модульного подхода, что обеспечивает удобство сопровождения и масштабируемость архитектуры.

Контроллер, отвечающий за добавление абонемента, принимает HTTP-запрос с названием, периодом валидности, временем посещения, ценой, услугами. Проецирует данные на структуры. Валидирует данные. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод Send у сервиса Abonement, который руководит добавлением абонемента. В результате выполнения контроллера возвращаются добавленный абонемент.

Контроллер, отвечающий за обновление абонемента, HTTP-запрос с названием, периодом валидности, временем посещения, ценой, услугами. Проецирует данные на структуры. Валидирует данные. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод Send у сервиса Abonement, который руководит обновлением абонемента. В результате выполнения контроллера возвращаются обновленный абонемент.

Он принимает HTTP-запрос с названием, периодом валидности, временем посещения, ценой, услугами. Проецирует данные на структуры. Валидирует данные. Создает запросы в формате Protobuf для дальнейшей передачи сервису Abonement, который руководит удалением абонемента.

3.10.3 Редактирование тренеров

Администратор системы имеет возможность редактирования тренеров: добавление, обновление, удаление.

Контроллер, отвечающий за добавление тренеров, принимает HTTP-запрос. Достает из контекста запроса структуру с данными для создания тренера. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод Send у сервиса Coach, который руководит добавлением тренера. В результате возвращаются добавленный тренер.

Контроллер, отвечающий за обновление тренеров, принимает HTTP-запрос. Достает из контекста запроса структуру с данными для обновления тренера. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод Send у сервиса Coach, который руководит обновлением тренера. В результате выполнения контроллера возвращаются обновленный тренер с услугами.

Контроллер, отвечающий за удаление тренеров, принимает HTTP-запрос. Достает из контекста запроса id тренера для удаления. Создает запросы в формате Protobuf для дальнейшей передачи сервисам. Вызывает метод DeleteCoachById у сервиса Coach, который руководит обновлением тренера.

3.10.4 Удаление отзывов о тренерах

У администратора есть возможность удаления отзывов о тренерах. Эта возможность реализована в контроллере DeleteCoachReview.

Реализация этого контроллера приведена в листинге 3.7.

```
func (h *Handler) DeleteCoachReview(c *gin.Context) {
    id := c.Param("id") if id == "" {
        logger.ErrorLogger.Printf("id not setted")
        c.JSON(http.StatusBadRequest, gin.H{"error": "id not
setted"})
        return
    }
    idUUID, err := uuid.Parse(id)
    if err != nil {
        logger.ErrorLogger.Printf("id not uuid")
        c.JSON(http.StatusBadRequest, gin.H{"error": "id not
uuid"}) return
    } deleteReviewByIdRequest :=
&reviewGRPC.DeleteReviewByIdRequest{Id: idUUID.String()}
    resp, err :=
(*h.reviewClient).DeleteReviewById(c.Request.Context(),
deleteReviewByIdRequest)
    if err != nil {
        logger.ErrorLogger.Printf(err.Error())
        c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()}) return
    } c.JSON(http.StatusOK, ) }
```

Листинг 3.11 – Реализация контроллера DeleteCoachReview

Этот контроллер обращается к gRPC методу для удаления отзыва к тренеру.

3.11 Структура клиентской части

Клиентская часть приложения реализована с использованием компонентного подхода с внедрением глобальных состояний. Основная логика и элементы пользовательского интерфейса размещены в директории src. Директории представлены в таблице 3.16.

Таблица 3.16 – Основные директории проекта в папке src и их назначение

Директория	Назначение
components	Включает в себя переиспользуемые React-компоненты, предназначенные для создания элементов пользовательского интерфейса веб-приложения.
context	Содержит файлы, предназначенные для управления токенами доступа пользователя, и глобальными состояниями приложения.
services	Включает модуль для логики работы токенов.
states	содержит файлы, связанные с управлением состоянием приложения с использованием Redux Toolkit [67].
images	Хранит изображения
utils	содержит файлы кастомных ошибок

Таблица соответствия маршрутов и компонентов страниц представлена в таблице 3.17.

Таблица 3.17 – Маршруты и компоненты страниц

Компонент страницы	Маршрут	Роль	Назначение компонента
MainNavHome	/main/home	Администратор, клиент	Основная страница приложения, которая служит отправной точкой для навигации по другим разделам и предоставляет общую информацию о веб-приложении.
MainNavAbonnements	/main/abonnements	Администратор, клиент	Отображение информации о абонементах, с элементами для поиска сортировки, фильтрации.
MainNavProfile	/main/profile	Клиент	Отображение профиля клиента с историей покупок.
MainNavCoaches	/main/coaches	Администратор, клиента	Отображения краткой информации о тренере.
MainNavCoachDetailCard	/main/coaches/details	Администратор, клиента	Отображения полной информации о тренере, списка его отзывов, а так же модального окна для написания отзыва(доступного только клиенту).
MainNavAdminPanel	/adminPanel	Администратор	Отображение модальных окон для редактирования абонементов, тренеров, клиентов.
SignIn	/signin		Отображение формы входа.
SignUp	/signup		Отображение формы регистрации.
MainTrainingRoom	/training-Room/:trainingId/:coachId/:coachName/:client-Name	Тренер, Клиент	Отображение онлайн тренировки.

Помимо маршрутов и страниц, приложение включает множество компонентов, которые обеспечивают функциональность и удобство использования клиентской части.

В таблице 3.18 представлено описание всех остальных компонентов приложения и их назначение.

Таблица 3.18 – Описание компонентов

Компонент	Назначение
mainAbonnements	Используется для отображения списка карточек абонементов. Отображает элементы фильтрации, сортировки, поиска.
abonementCard	Используется для отображения карточки абонемента.
AbonnementsModal	Используется для отображения модального окна для редактирования абонементов.
ClientsModal	Используется для отображения модального окна для редактирования клиентов.
CoachesModal	Используется для отображения модального окна для редактирования тренеров.
mainCoaches	Используется для отображения списка карточек тренеров.
coachCard	Используется для отображения карточки тренера.
coachDetailCard	Используется для отображения карточки тренера с полной информацией, с комментариями и модальным окном для их написания, а также с расписанием тренеров со слотами для бронирования онлайн тренировки.
mainNav	Используется для отображения элемента с кнопками для навигации по всем страницам.
MainTrainingRoom	Отображение онлайн тренировки. Установка WebRTC соединения.

Эти компоненты обеспечивают все необходимые функции для создания полноценного приложения для фитнес-центра.

3.12 Подключение к онлайн тренировке

У клиентов и тренеров есть возможность подключения к онлайн тренировке. Эта возможность реализована в контроллере Join.

Он принимает HTTP-запрос с id комнаты, id тренера, id комнаты. Процирует данные на структуру для получения конкретного типа данных. Валидирует данные. Создает websocket подключение с клиентом для дальнейшего обмена данными, для установления WebRTC соединения. Код клиентской части представлен в приложении Д.

3.13 Выводы по разделу

Разработано веб-приложение для фитнес-центра на Golang с использованием GIN и gRPC. В качестве СУБД использована PostgreSQL. Архитектура – микросервисная. Интегрированы сторонние сервисы для оплаты, облачного хранения и реализован набор функций в размере 26.